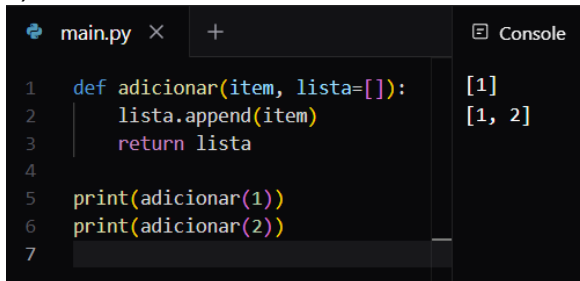


Exercício em duplas: qual é a saída?

Para cada trecho: (a) qual é a saída? (b) qual conceito da aula explica?

1 - Python:

a)



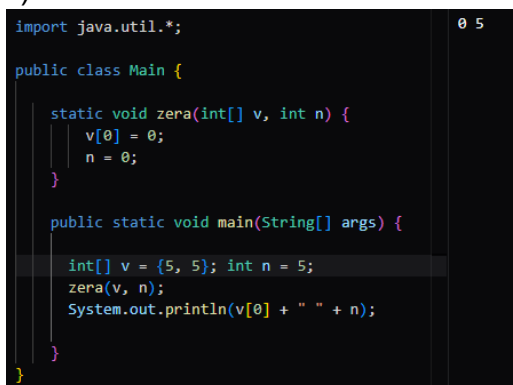
```
main.py x + Console
1 def adicionar(item, lista=[]):
2     lista.append(item)
3     return lista
4
5 print(adicionar(1))
6 print(adicionar(2))
7
```

The console shows the output: [1] and [1, 2].

b) Parâmetro padrão e passagem por atribuição.

2 - Java:

a) Saída: 0 5



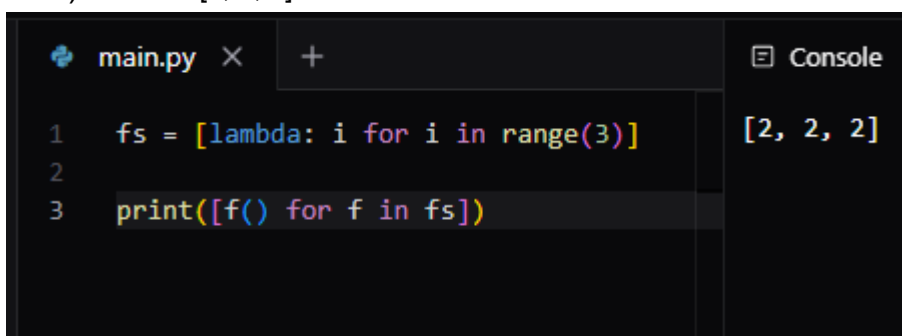
```
import java.util.*;
public class Main {
    static void zera(int[] v, int n) {
        v[0] = 0;
        n = 0;
    }
    public static void main(String[] args) {
        int[] v = {5, 5}; int n = 5;
        zera(v, n);
        System.out.println(v[0] + " " + n);
    }
}
```

The console shows the output: 0 5.

b) O array é um objeto (a cópia da referência permite alterar v[0]), e o int é copiado. Conceito: Java passa tudo por valor - slide 11

3 - Python:

a) Saída: [2, 2, 2]



```
main.py x + Console
1 fs = [lambda: i for i in range(3)]
2
3 print([f() for f in fs])
```

The console shows the output: [2, 2, 2].

b) O conceito apresentado é fechamentos

4 - C:

a)



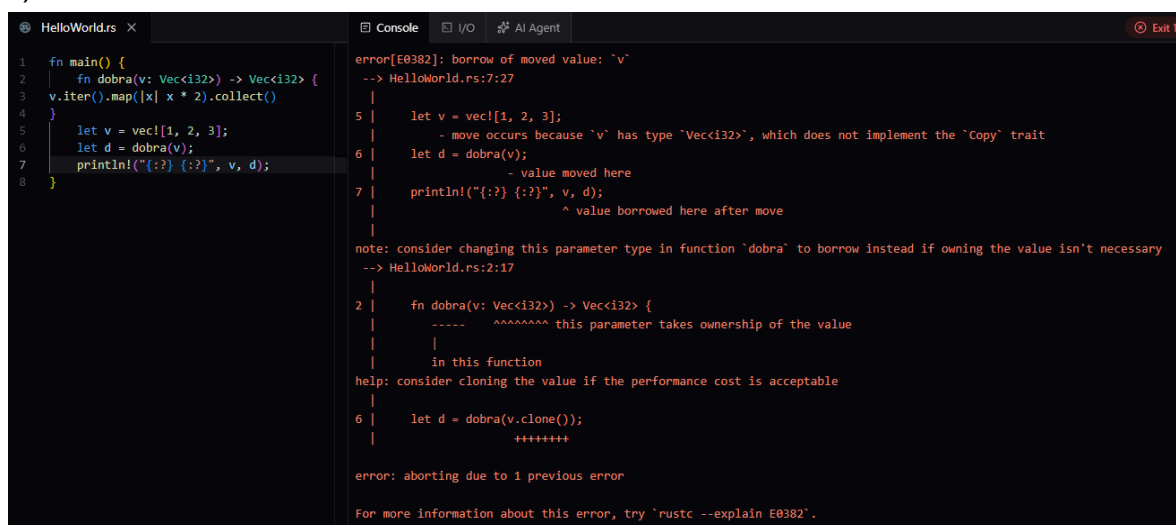
```
1  #include <stdio.h>
2
3  int contador(void) {
4      static int n = 0;
5      return ++n;
6  }
7
8  int main()
9  {
10     contador(); contador();
11     printf("%d\n", contador());
12
13 }
```

The screenshot shows a C program in a code editor. The program defines a function `contador` with a static integer variable `n` initialized to 0. The function increments `n` and returns its value. In the `main` function, `contador` is called twice, and then its return value is printed using `printf`. The output in the console is 3.

b) O código utiliza o conceito de variável local estática (static). A variável `n` mantém seu valor entre as chamadas da função `contador`. Por isso, após três chamadas, o programa imprime 3.

5 - Rust:

a)



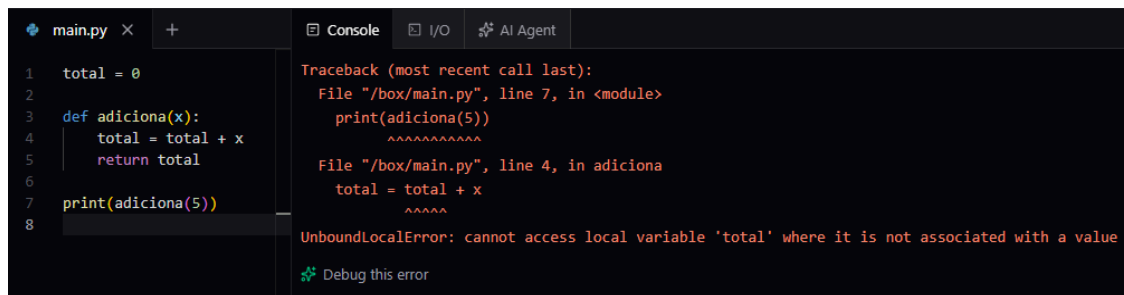
```
1  fn main() {
2      fn dobra(v: Vec<i32>) -> Vec<i32> {
3          v.iter().map(|x| x * 2).collect()
4      }
5      let v = vec![1, 2, 3];
6      let d = dobra(v);
7      println!("{:?} {:?}", v, d);
8  }
```

The screenshot shows a Rust program in a code editor. The program defines a function `dobra` that takes a `Vec<i32>` and returns a `Vec<i32>`. In the `main` function, a vector `v` is created, `dobra` is called with `v`, and the result `d` is printed. The console shows an error: `error[E0382]: borrow of moved value: 'v'`. The error message explains that `v` is moved into the `dobra` function, so it cannot be used again in the `println!` statement. The console also provides a note suggesting to change the parameter type in `dobra` to borrow instead of own the value.

b) O `v` foi passado para a função `dobra` e depois foi usado novamente. Em Rust, isso não é permitido.

6 - Python:

a)



```
main.py x + Console I/O AI Agent
1 total = 0
2
3 def adiciona(x):
4     total = total + x
5     return total
6
7 print(adiciona(5))
8

Traceback (most recent call last):
  File "/box/main.py", line 7, in <module>
    print(adiciona(5))
    ^^^^^^^^^
  File "/box/main.py", line 4, in adiciona
    total = total + x
    ^^^^^
UnboundLocalError: cannot access local variable 'total' where it is not associated with a value
Debug this error
```

b) O conceito é ambiente de referenciamento local e escopo